Collective Communication

Why Do We Use Collective Communication?

- Instead of sending the same data using multiple point-to-point messages
- This approach becomes complex and inefficient as the number of processes increases
- Collective communication allows a **single call** to involve all processes
- For example, a **broadcast** sends data from one process to all others in one operation
- MPI internally uses optimized and scalable communication algorithms

Types of Collective Communication

Collective operations are classified into **two categories**

Data movement collectives

- Used to distribute or collect data among processes
- Examples: Broadcast, Scatter, Gather, Allgather.

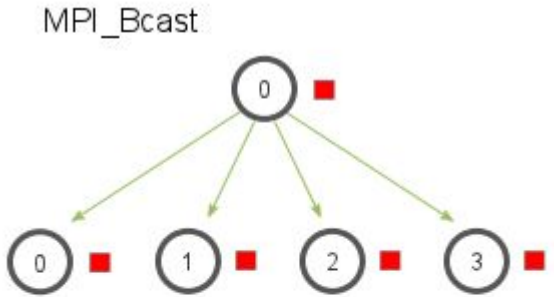
Collective computation

- Used to compute a global result from all processes
- Examples: Reduce, Allreduce.

MPI_Bcast

MPI_Bcast(buffer, count, datatype, root, comm)

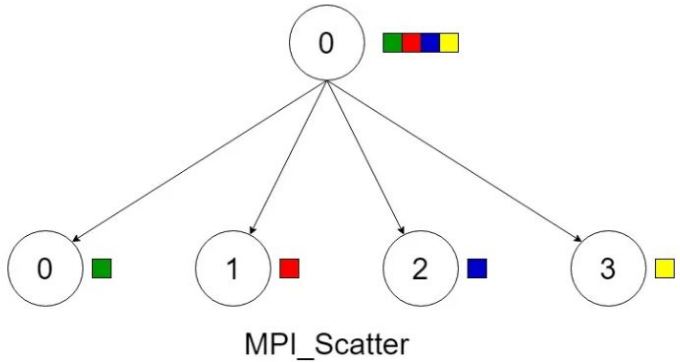
- **buffer** – address of data to send
- **count** – number of elements in the buffer
- **datatype** – type of each element (e.g., MPI_INT, MPI_FLOAT)
- **root** – rank of the broadcasting process
- **comm** – communicator including all participating processes



MPI_Scatter

MPI_Scatter(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm)

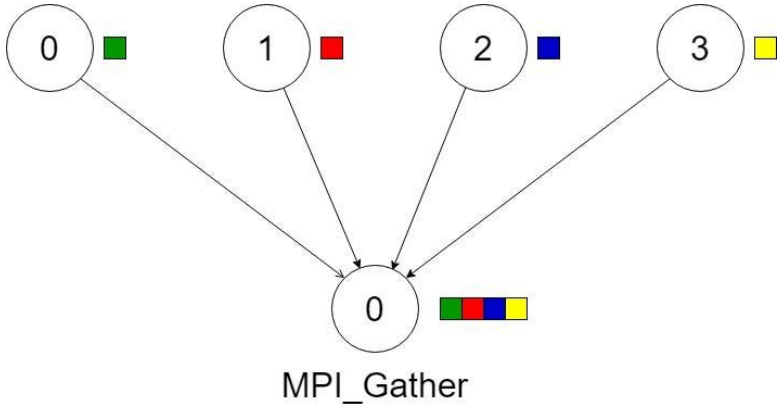
- **sendbuf** – buffer containing data to be scattered (root only)
- **sendcount** – number of elements sent to each process
- **sendtype** – datatype of send buffer elements
- **recvbuf** – buffer to receive data on each process
- **recvcount** – number of elements received by each process
- **recvtype** – datatype of receive buffer elements
- **root** – rank of the sending process
- **comm** – communicator over which scatter occurs



MPI_Gather

MPI_Gather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, root, comm)

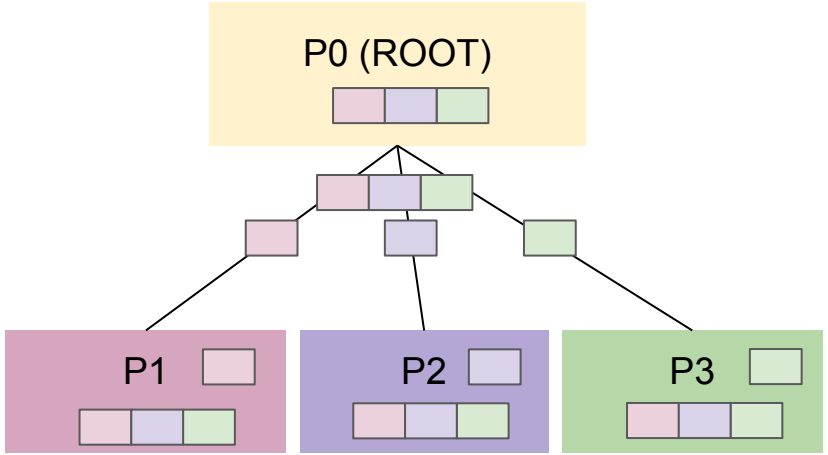
- **sendbuf** – buffer containing data sent by each process
- **sendcount** – number of elements sent by each process
- **sendtype** – datatype of send buffer elements
- **recvbuf** – buffer where root collects data
- **recvcount** – number of elements received from each process
- **recvtype** – datatype of receive buffer elements
- **root** – rank of the collecting process
- **comm** – communicator over which gather occurs



MPI_Allgather

MPI_Allgather(sendbuf, sendcount, sendtype, recvbuf, recvcount, recvtype, comm)

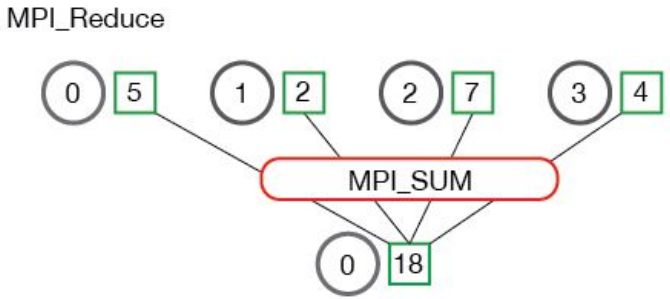
- **sendbuf** – buffer containing data from each process
- **sendcount** – number of elements sent by each process
- **sendtype** – datatype of send buffer elements
- **recvbuf** – buffer where all gathered data is stored
- **recvcount** – number of elements received from each process
- **recvtype** – datatype of receive buffer elements
- **comm** – communicator over which allgather occurs



MPI_Reduce

MPI_Reduce(sendbuf, recvbuf, count, datatype, op, root, comm)

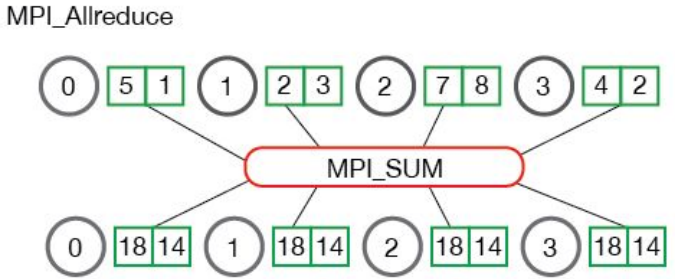
- **sendbuf** – buffer containing local data from each process
- **recvbuf** – buffer where reduced result is stored (root only)
- **count** – number of elements to be reduced
- **datatype** – datatype of elements
- **op** – reduction operation (e.g., MPI_SUM, MPI_MAX)
- **root** – rank of the process receiving the result
- **comm** – communicator over which reduction occurs



MPI_Allreduce

MPI_Allreduce(sendbuf, recvbuf, count, datatype, op, comm)

- **sendbuf** – buffer containing local data from each process
- **recvbuf** – buffer where final reduced result is stored on all processes
- **count** – number of elements to be reduced
- **datatype** – datatype of elements
- **op** – reduction operation (e.g., MPI_SUM, MPI_MAX)
- **comm** – communicator over which allreduce occurs



MPI_Barrier

MPI_Barrier(comm)

- Synchronizes all processes in a communicator
- Ensures no process continues until all have reached the barrier
- Used to coordinate phases of parallel execution

Questions

1. Write an MPI program in which rank 0 initializes an integer array and distributes it to all processes using ***MPI_Bcast***. Each process should print the received array.
2. Rewrite Q1 using multiple ***MPI_Send*** and ***MPI_Recv*** calls instead of ***MPI_Bcast***. Compare the two implementations in terms of code complexity and execution time.
3. Each process generates an integer equal to its rank. Collect all values at rank 0 using ***MPI_Gather*** and print the gathered array at the root.
4. Modify Q3 to use ***MPI_Allgather*** instead of ***MPI_Gather***. Print the gathered array on every process.
5. Each process computes a local sum of elements of an array. Use ***MPI_Reduce*** with ***MPI_SUM*** to compute the global sum at rank 0.
6. Rewrite Q5 using MPI_Allreduce. Print the final result on all processes.
7. Write an MPI program in which process 0 initializes an integer array of size N and distributes **equal-sized contiguous blocks** of this array to all processes using ***MPI_Scatter***. Each process should print the portion of the array it receives.
8. Write an MPI program that uses a **non-blocking collective communication** operation (***MPI_Ibcast, MPI_Igather, MPI_Iscatter and MPI_Iallreduce***). Complete the operation using ***MPI_Wait*** and print the result after the communication finishes.

Application based Question (Refer book)

Design an MPI program to estimate π using the Monte Carlo method with the following approach:

- Use **P MPI processes**; the **last process acts as a random number server**, others are workers.
- Rank 0 reads the accuracy ϵ and broadcasts it to all processes.
- Workers repeatedly request random numbers from the server in fixed-size chunks.
- Each worker maps random numbers to (x,y) points and counts **inside** and **outside** the unit circle.
- Use **MPI_Allreduce** to compute global counts and estimate π.
- Terminate when the error is less than ϵ (or a maximum number of points is reached).

Write code for the server and worker processes.

THANK YOU